ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403
anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450
phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông
Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhẫn _____
Chữ ký của GVHD

KHOA: *(Điền tên khoa)*

Hà Nội, 01/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng <...> năm <...>

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT **4**

DANH SÁCH HÌNH VẼ **6**

DANH SÁCH BẢNG **7**

PHÂN CHIA CÔNG VIỆC **8**

MỞ ĐẦU **9**
Bối cảnh và vấn đề . . . 9
Mục tiêu và phạm vi . . . 9
Deliverables . . . 9
Cấu trúc báo cáo . . . 10

1 TỔNG QUAN HỆ THỐNG VÀ CƠ SỞ LÝ THUYẾT **11**
1.1 Kiến trúc hệ thống . . . 11
1.1.1 Zigbee Network (2.4 GHz) . . . 11
1.1.2 Coordinator / NCP (Network Co-Processor) . . . 12
1.1.3 Giao tiếp Host – NCP (EZSP/ASH qua UART) . . . 13
1.1.4 Gateway Host (Z3Gateway trên Linux) . . . 13
1.1.5 Cloud Backend . . . 14
1.1.6 Flutter Mobile App . . . 14
1.1.7 Use Cases theo vai trò . . . 15
1.2 Giao thức Zigbee . . . 15
1.2.1 Tổng quan . . . 15
1.2.2 Vai trò thiết bị . . . 15
1.2.3 Topology mạng . . . 15
1.2.4 Kiến trúc Protocol Stack . . . 17
1.2.5 Địa chỉ và Bảo mật . . . 18
1.2.6 Quá trình hình thành và gia nhập mạng . . . 18
1.3 Zigbee Cluster Library (ZCL) . . . 19
1.3.1 Cấu trúc ZCL . . . 20
1.3.2 Binding (Liên kết) . . . 20
1.3.3 Group (Nhóm) . . . 21
1.4 Giao thức MQTT . . . 21
1.4.1 Tổng quan . . . 21
1.4.2 Thành phần chính . . . 22
1.4.3 Quality of Service (QoS) . . . 22
1.4.4 Retained Messages và Last Will . . . 22
1.4.5 Topics và Wildcards . . . 23
1.5 Giao tiếp UART . . . 23
1.5.1 Tổng quan . . . 23
1.5.2 Frame Format . . . 23
1.5.3 Ứng dụng UART trong dự án: EZSP/ASH . . . 24

1.6 Phần cứng và Kit sử dụng . . . 24
1.6.1 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit . . . 24
1.6.2 Cấu hình phần cứng theo thiết bị . . . 25
1.6.3 Toolchain và SDK . . . 26
1.7 Luồng dữ liệu (Data Flow) . . . 26
1.7.1 Luồng Uplink (State/Event Report) . . . 26
1.7.2 Luồng Downlink (Command) . . . 26
1.7.3 Luồng Local Control . . . 27
1.8 MQTT Contract . . . 27
1.8.1 Namespace và Topic Structure . . . 27
1.8.2 Message Envelope . . . 28
1.8.3 Retain và QoS Policy . . . 28

2 THIẾT KẾ HỆ THỐNG 30
2.1 Yêu cầu chức năng và phi chức năng . . . 30
2.1.1 Yêu cầu chức năng . . . 30
2.1.2 Yêu cầu phi chức năng . . . 30
2.2 Network và Device Roles (Zigbee) . . . 31
2.3 Data và Message Design . . . 32
2.3.1 MQTT Message Channels . . . 32
2.3.2 State Message Design . . . 32
2.3.3 Command Message Design . . . 33
2.3.4 Versioning Rule . . . 33
2.4 Logic Flow và State Machine . . . 34
2.4.1 Light Device State Machine . . . 34
2.4.2 Command Lifecycle . . . 34
2.4.3 Gateway Runtime Flow . . . 35
2.5 Cloud Backend Architecture . . . 35
2.5.1 Kiến trúc tổng quan . . . 35
2.5.2 Database Design . . . 35
2.5.3 REST API Endpoints . . . 36
2.5.4 MQTT – Database Integration . . . 36
2.6 Trade-offs và Quyết định thiết kế . . . 36

3 TRIỂN KHAI HỆ THỐNG 38
3.1 Triển khai Firmware . . . 38
3.1.1 Cấu trúc Firmware Coordinator . . . 38
3.1.2 Firmware Light Device . . . 39
3.1.3 Firmware Switch Device . . . 39
3.1.4 Firmware Occupancy Sensor . . . 39
3.2 Triển khai Gateway Service . . . 40
3.2.1 Thiết lập môi trường . . . 40
3.2.2 Kiến trúc đa luồng . . . 40
3.2.3 Triển khai UART Line Protocol . . . 40
3.2.4 Cơ chế Correlation ID . . . 41
3.2.5 Chính sách Timeout và Retry . . . 41
3.2.6 Rules Engine . . . 41
3.2.7 Admin API . . . 42
3.3 Triển khai Cloud (Firebase) . . . 42

3.3.1 Firebase Setup 42
3.3.2 Device State Sync 42
3.3.3 Event History và Logging 42
3.4 Triển khai Flutter Mobile App 42
3.4.1 App Architecture 42
3.4.2 Device Control UI 42
3.4.3 Realtime State Display 42
3.4.4 Occupancy Monitoring 43
3.5 Luồng End-to-end Command/State 43
3.5.1 App → Cloud → Gateway → Light 43
3.5.2 Local Automation: Occupancy → Light 43
3.6 Hướng phát triển OTA 43

4 KẾT QUẢ VÀ ĐÁNH GIÁ 44
4.1 Điều kiện chạy thử (Test Setup) 44
4.2 Kết quả End-to-end 44
4.3 Kết quả từng thành phần 45
4.3.1 Zigbee Network 45
4.3.2 Gateway Service 45
4.3.3 Cloud Firebase 45
4.3.4 Flutter Mobile App 45
4.4 Đánh giá hiệu năng 45

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 46
5.1 Kết luận 46
5.2 Hạn chế 46
5.3 Hướng phát triển 47

TÀI LIỆU THAM KHẢO 48

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-The-Air (firmware update)
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1 Kiến trúc tổng quan hệ thống Smart Home Platform 11
1.2 Star topology – Coordinator ở trung tâm 16
1.3 Mesh topology – Định tuyến qua nhiều node 16
1.4 Tree topology – Cấu trúc phân cấp 17
1.5 Kiến trúc Zigbee Protocol Stack 17
1.6 Mô hình Client/Server trong ZCL 20
1.7 Kiến trúc Publish/Subscribe của MQTT 21
1.8 Kết nối UART giữa hai thiết bị 23
1.9 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit 24
1.10 Luồng dữ liệu Uplink 26
1.11 Luồng điều khiển Downlink 26

DANH SÁCH BẢNG

2 Bảng phân chia công việc 8

1.1 Các mức QoS trong MQTT 22
1.2 Toolchain và SDK version 26
1.3 Các nhóm MQTT topic chính 28

2.1 Thiết kế Zigbee Device Roles 31
2.2 Thiết kế các kênh MQTT 32
2.3 Các bảng database chính 36
2.4 REST API endpoints chính 36

3.1 Coordinator Firmware – Ánh xạ module 38
3.2 Light Device Firmware – Cấu trúc 39
3.3 Switch Device Firmware – Cấu trúc 39
3.4 Occupancy Sensor Firmware – Cấu trúc 39
3.5 Công cụ và phiên bản sử dụng 40
3.6 Tham số Timeout và Retry 41
3.7 Admin API Endpoints 42

PHÂN CHIA CÔNG VIỆC

STT	Công việc	Sinh viên thực hiện
Firmware – Zigbee Devices		
1	<Firmware Coordinator: tạo mạng, Trust Center, ZCL, UART bridge>	<SV>
2	<Firmware Light Device: On/Off Server, LED indicator, state report>	<SV>
3	<Firmware Switch Device: On/Off Client, binding, button control>	<SV>
4	<Firmware Occupancy Sensor: occupancy detection, ZCL report>	<SV>
Gateway Service (Ubuntu / Raspberry Pi)		
5	<Gateway UART module: kết nối COM, đọc line, reconnect>	<SV>
6	<Gateway MQTT bridge: subscribe cmd, publish state/telemetry/ack>	<SV>
7	<Gateway rules engine, ACK routing, retry/timeout>	<SV>
8	<Gateway Admin API (/health, /status, /config, /rules)>	<SV>
Cloud – Firebase		
9	<Firebase setup: Realtime DB / Firestore schema, Cloud Functions>	<SV>
10	<MQTT – Firebase bridge: device state sync, command relay>	<SV>
11	<Event history, logging, device management trên cloud>	<SV>
Mobile App – Flutter		
12	<Flutter app: device list, light on/off control UI>	<SV>
13	<Flutter app: occupancy state display, realtime update>	<SV>
14	<Flutter app: event history, notification>	<SV>
Tổng hợp & Tài liệu		
15	<Tổng hợp và hoàn thiện báo cáo>	<SV 1, SV 2>

Bảng 2: Bảng phân chia công việc

MỞ ĐẦU

1. Bối cảnh và vấn đề

Trong bối cảnh smart home (nhà thông minh) và smart building (tòa nhà thông minh) ngày càng phổ biến, nhu cầu xây dựng hệ thống IoT hoàn chỉnh từ tầng thiết bị đến ứng dụng di động trở thành một bài toán có giá trị cả về mặt kỹ thuật lẫn thực tiễn. Các giải pháp hiện có trên thị trường thường là hệ sinh thái đóng, khó tùy biến và mở rộng. Việc nghiên cứu và xây dựng một nền tảng mở, dựa trên các chuẩn giao tiếp phổ biến như Zigbee và MQTT, cho phép kiểm soát toàn bộ luồng dữ liệu từ thiết bị đến người dùng.

Zigbee được lựa chọn cho tầng local network bởi giao thức này phù hợp với mô hình nhiều thiết bị nhỏ gọn, tiết kiệm năng lượng, hỗ trợ mesh network giúp tăng phạm vi phủ sóng và độ tin cậy. Zigbee đã được ứng dụng rộng rãi trong smart home với các thiết bị chiếu sáng, công tắc, cảm biến hiện diện (occupancy sensor) và nhiều loại thiết bị khác.

2. Mục tiêu và phạm vi

Đề tài hướng tới xây dựng một nền tảng smart home mini-platform dựa trên kiến trúc IoT nhiều lớp, bao gồm:

- **Local network:** Mạng Zigbee kết nối các thiết bị: đèn thông minh (light device), công tắc (switch device), cảm biến hiện diện (occupancy sensor), được điều phối bởi Zigbee Coordinator.
- **Gateway:** Chạy trên Linux (Ubuntu, hướng migrate sang Raspberry Pi), đóng vai trò bridge giữa mạng Zigbee và cloud thông qua UART và MQTT.
- **Cloud:** Sử dụng Firebase cho backend, quản lý device state, command dispatch, event history.
- **Mobile App:** Ứng dụng Flutter hiển thị trạng thái thiết bị, điều khiển bật/tắt đèn, xem occupancy state, nhận cập nhật realtime.

Phạm vi triển khai tập trung vào kiểm chứng luồng dữ liệu end-to-end: App gửi command → Cloud nhận → Gateway dịch sang Zigbee command → Light Device thực thi → State report quay lại Cloud và App.

3. Deliverables

1. Firmware cho 4 node Zigbee: Coordinator, Light Device, Switch Device, Occupancy Sensor.
2. Gateway service chạy trên Ubuntu (Python), bridge UART ↔ MQTT.
3. Cloud backend trên Firebase (Realtime DB / Firestore, Cloud Functions).
4. Ứng dụng di động Flutter với chức năng điều khiển đèn và giám sát occupancy.
5. Tài liệu kỹ thuật: đặc tả giao thức, MQTT topics, kiến trúc hệ thống.

4. Cấu trúc báo cáo

Báo cáo được tổ chức thành 5 chương:

- **Chương 1 – Tổng quan hệ thống và cơ sở lý thuyết:** Trình bày kiến trúc hệ thống, các thành phần, phần cứng sử dụng, và nền tảng lý thuyết Zigbee, ZCL, MQTT, UART.

CHƯƠNG 1
TỔNG QUAN HỆ THỐNG VÀ CƠ SỞ LÝ THUYẾT

Chương này trình bày kiến trúc tổng thể và các thành phần của hệ thống Smart Home Platform. Nội dung bao gồm: (1) Kiến trúc hệ thống end-to-end theo mô hình NCP (Network Co-Processor) với các end device Zigbee (Light, Switch, Occupancy Sensor), Coordinator/NCP radio, Z3Gateway host trên Linux, Cloud Backend (FastAPI) và Flutter Mobile App; (2) Lý thuyết nền tảng Zigbee, ZCL, MQTT, UART; (3) Phần cứng và Kit sử dụng; (4) Luồng dữ liệu Uplink/Downlink; (5) Đặc tả MQTT Contract.

1.1 Kiến trúc hệ thống

Hệ thống Smart Home Platform được thiết kế theo kiến trúc IoT nhiều lớp (multi-layer), trong đó mỗi lớp đảm nhận một vai trò rõ ràng. Luồng dữ liệu end-to-end đi qua các lớp sau:

End Devices (Zigbee RF) $\longleftrightarrow$ *NCP Radio (EFR32)* $\longleftrightarrow$ *Z3Gateway Host (Linux)* $\longleftrightarrow$ *MQTT Broker* $\longleftrightarrow$ *Cloud Backend / Mobile App*

Đặc điểm quan trọng: Cloud và Mobile App không giao tiếp trực tiếp với radio Zigbee. Gateway host là điểm hội tụ duy nhất giữa thế giới IP và mạng Zigbee RF.

Hình 1.1: Kiến trúc tổng quan hệ thống Smart Home Platform

1.1.1 Zigbee Network (2.4 GHz)

Hệ thống mạng không dây đóng vai trò cốt lõi, sử dụng chuẩn Zigbee 3.0 hoạt động ở tần số 2.4 GHz. Mạng Zigbee bao gồm các end device sau:

a. **Light Device** – Router (bộ định tuyến)

- **Vai trò:** Nhận lệnh điều khiển bật/tắt từ Coordinator hoặc Switch và phản hồi trạng thái.
- **Thành phần bên trong:**
  - *MCU EFR32:* Vi điều khiển chạy firmware điều khiển đèn.
  - *LED indicator:* Phản ánh trạng thái đèn (ON/OFF).
  - *Zigbee Router stack:* Duy trì kết nối mạng, hỗ trợ routing.
  - *On/Off Server Cluster:* Lưu trữ và quản lý trạng thái bật/tắt.
- **Luồng hoạt động:**
  - Nhận ZCL On/Off Command từ Coordinator hoặc Switch (qua Binding).
  - Cập nhật trạng thái LED tương ứng.
  - Gửi ZCL Default Response / Report Attributes xác nhận trạng thái.

b. **Switch Device** – Router (bộ định tuyến)

- **Vai trò:** Gửi lệnh bật/tắt đèn khi người dùng nhấn nút, hỗ trợ local control không qua cloud.
- **Thành phần bên trong:**
  - *MCU EFR32:* Vi điều khiển chạy firmware công tắc.
  - *Button (PB0/PB1):* Nút nhấn để toggle đèn.
  - *Zigbee Router stack:* Duy trì kết nối mạng.
  - *On/Off Client Cluster:* Gửi lệnh On/Off/Toggle.
- **Luồng hoạt động:**
  - Người dùng nhấn nút → Switch gửi ZCL Toggle Command.
  - Command được gửi tới Light Device qua Binding hoặc qua Coordinator.

c. **Occupancy Sensor** – End Device (thiết bị đầu cuối)

- **Vai trò:** Phát hiện sự hiện diện của người và gửi trạng thái lên Coordinator.
- **Thành phần bên trong:**
  - *MCU EFR32:* Vi điều khiển chạy firmware cảm biến.
  - *Occupancy sensing:* Đầu vào phát hiện hiện diện (mô phỏng bằng button hoặc sensor thực).
  - *Zigbee End Device stack:* Tham gia mạng và gửi report.
  - *Occupancy Sensing Cluster (Server):* Lưu trạng thái occupied/unoccupied.
  - *Power mode:* Sleepy End Device (SED) tiết kiệm năng lượng.
- **Luồng hoạt động:**
  - Phát hiện thay đổi trạng thái occupancy → gửi ZCL Report Attributes lên Coordinator.
  - Coordinator chuyển tiếp trạng thái lên Gateway host qua EZSP.

1.1.2 Coordinator / NCP (Network Co-Processor)

Dự án sử dụng mô hình NCP (Network Co-Processor), trong đó chip EFR32MG12 chạy firmware NCP xử lý toàn bộ Zigbee protocol stack (PHY, MAC, NWK, APS), còn application logic chạy trên máy tính host (Linux). Đây là mô hình được Silicon Labs khuyến nghị cho các gateway có tài nguyên host dồi dào.

- **NCP radio (EFR32MG12):** Chạy firmware NCP (EmberZNet), xử lý Zigbee stack, đóng vai trò Coordinator và Trust Center của mạng.
- **Chức năng:**
  - Hình thành mạng Zigbee (form network), quản lý PAN.
  - Cho phép thiết bị gia nhập (permit joining).
  - Quản lý bảo mật: Trust Center, Network Key, Link Key.
  - Thu nhận ZCL Report từ end device, chuyển tiếp lên host qua EZSP.
  - Nhận lệnh ZCL từ host và phát xuống end device qua RF.

- **Giao tiếp với host:** Qua UART (USB Virtual COM Port), sử dụng giao thức EZSP (EmberZNet Serial Protocol) với framing ASH (Asynchronous Serial Host).

So sánh mô hình NCP và SoC:

- *Mô hình SoC:* Application và Zigbee stack chạy chung trên một MCU. Phù hợp cho thiết bị đơn giản (light, switch, sensor).
- *Mô hình NCP:* Zigbee stack chạy trên radio chip, application chạy trên host riêng. Phù hợp cho gateway cần tài nguyên lớn (MQTT, database, cloud integration).

Trong dự án này, các end device (Light, Switch, Sensor) chạy mô hình SoC, còn Coordinator sử dụng mô hình NCP với host application là Z3Gateway trên Linux.

1.1.3 Giao tiếp Host – NCP (EZSP/ASH qua UART)

Kênh truyền dữ liệu giữa Z3Gateway host và NCP radio sử dụng kết nối UART (USB Virtual COM Port) với giao thức EZSP/ASH do Silicon Labs định nghĩa.

- **EZSP (EmberZNet Serial Protocol):** Giao thức lớp ứng dụng cho phép host gửi lệnh điều khiển Zigbee stack trên NCP (form network, send ZCL command, read attribute...) và nhận callback/event từ NCP (device joined, attribute report, message received...).
- **ASH (Asynchronous Serial Host):** Lớp framing đảm bảo truyền tin cậy trên UART, cung cấp cơ chế ACK, retry, CRC và flow control. ASH đóng gói các EZSP frame thành các byte stream truyền qua serial.
- **Đặc tả kết nối vật lý:**
  - Kết nối: USB Virtual COM Port (qua J-Link debugger trên Mainboard BRD4001A).
  - Cấu hình: 115200 baud, 8N1.

Điểm quan trọng: dự án **không** định nghĩa giao thức UART tùy chỉnh riêng trên biên host–radio. Toàn bộ giao tiếp host–NCP sử dụng EZSP/ASH chuẩn từ Silicon Labs SDK, đảm bảo tương thích và ổn định.

1.1.4 Gateway Host (Z3Gateway trên Linux)

Z3Gateway là ứng dụng host viết bằng ngôn ngữ C, dựa trên Z3Gateway sample application của Silicon Labs Gecko SDK. Ứng dụng chạy trên máy tính Linux (Ubuntu), đóng vai trò bridge giữa mạng Zigbee (qua NCP) và hệ thống cloud/app (qua MQTT).

- **Kiến trúc runtime:** Single-process, tích hợp trực tiếp MQTT client trong cùng tiến trình C với EZSP host logic.
- **Các module chính:**
  - `app_mqtt`: Quản lý kết nối MQTT broker, publish/subscribe.
  - `cmd_handler`: Xử lý command nhận từ MQTT, chuyển thành ZCL command gửi xuống NCP.
  - `device_monitor`: Theo dõi danh sách thiết bị trong mạng, trạng thái join/leave.

  - `telemetry_rx`: Nhận và xử lý ZCL Report Attributes từ NCP, chuẩn hóa thành message publish lên MQTT.
  - `net_mgr`: Quản lý trạng thái mạng Zigbee (form, open, close).
  - `app_state`: Quản lý trạng thái nội bộ của gateway.

- **Vai trò trong hệ thống:**
  - Bridge dữ liệu Zigbee ↔ MQTT.
  - Chuẩn hóa (normalize) dữ liệu từ ZCL sang định dạng JSON qua MQTT.
  - Theo dõi sức khỏe thiết bị (device registry, health monitoring).
  - Ghi log sự kiện quan trọng.

1.1.5 Cloud Backend

Cloud Backend là thành phần phía máy chủ, cung cấp dịch vụ lưu trữ, quản lý và API cho toàn bộ hệ thống.

- **Framework:** FastAPI (Python) – web framework hiệu năng cao, hỗ trợ async.
- **Database:** SQLAlchemy ORM với SQLite (giai đoạn phát triển), hướng tới PostgreSQL cho production.
- **Tích hợp MQTT:** Cloud backend subscribe các topic từ broker để nhận dữ liệu state/event từ gateway, đồng thời publish command request khi người dùng điều khiển từ app.
- **Database tables:**
  - `devices`: Danh sách thiết bị đã đăng ký (device_id, device_type, eui64, nwk_addr...).
  - `device_states`: Trạng thái hiện tại của thiết bị (on/off, occupancy...).
  - `events`: Lịch sử sự kiện (state change, join/leave, error...).
  - `commands`: Lịch sử lệnh điều khiển và trạng thái thực thi.
- **REST API endpoints:** health check, quản lý devices, truy vấn events, gửi commands.

1.1.6 Flutter Mobile App

- **Vai trò:** Giao diện người dùng trên thiết bị di động, đóng vai trò presentation layer của hệ thống.
- **Chức năng:**
  - Hiển thị danh sách thiết bị và trạng thái realtime.
  - Điều khiển bật/tắt đèn (Light Device).
  - Xem trạng thái occupancy.
  - Xem lịch sử sự kiện.
- **Giao tiếp:** Thông qua REST API của Cloud Backend hoặc trực tiếp qua MQTT broker.

1.1.7 Use Cases theo vai trò

- **User (cư dân):** Bật/tắt đèn từ app, xem trạng thái thiết bị, nhận thông báo occupancy, điều khiển local qua Switch.
- **Admin (quản trị viên):** Cấu hình Gateway, quản lý thiết bị join/leave, theo dõi health/log, thiết lập automation rules.

1.2 Giao thức Zigbee

1.2.1 Tổng quan

Zigbee là tiêu chuẩn giao tiếp không dây được phát triển bởi Connectivity Standards Alliance (CSA), đặc trưng bởi chi phí thấp và tiêu thụ năng lượng cực thấp. Zigbee hoạt động chủ yếu trên dải tần 2.4 GHz (toàn cầu) với tốc độ 250 kbps.

Đặc điểm chính:

- Xây dựng trên nền tảng IEEE 802.15.4 – IEEE là Institute of Electrical and Electronics Engineers (Viện Kỹ sư Điện và Điện tử) – định nghĩa lớp PHY (Physical – vật lý) và MAC (Media Access Control – điều khiển truy cập môi trường)
- Hỗ trợ mạng Star và Mesh topology (cấu hình mạng)
- Bảo mật AES-128 bit – AES là Advanced Encryption Standard (chuẩn mã hóa nâng cao) – với Trust Center
- Phù hợp cho IoT, tự động hóa nhà và công nghiệp

1.2.2 Vai trò thiết bị

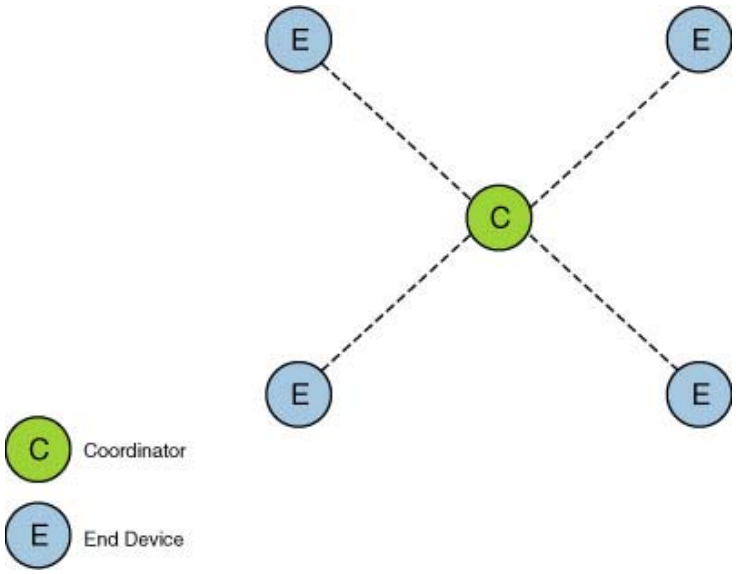
- **Zigbee Coordinator (ZC):** Khởi tạo và quản lý mạng, đóng vai trò Trust Center, địa chỉ luôn là `0x0000`. Mỗi mạng chỉ có một Coordinator.
- **Zigbee Router (ZR):** Định tuyến dữ liệu, mở rộng phạm vi mạng. Luôn bật, không sleep.
- **Zigbee End Device (ZED):** Thiết bị đầu cuối tiết kiệm năng lượng, chỉ giao tiếp qua thiết bị cha, có thể sleep.

1.2.3 Topology mạng

Zigbee hỗ trợ nhiều mô hình mạng khác nhau tùy theo yêu cầu ứng dụng:

a) Star Topology (Mạng hình sao):

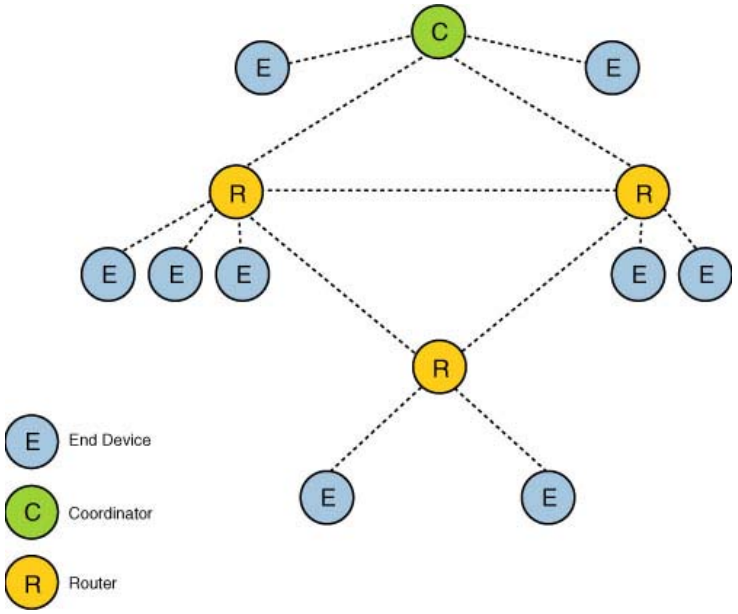
Mạng đơn giản nhất với Coordinator ở trung tâm, các End Device kết nối trực tiếp. Ưu điểm là dễ triển khai, End Device có thể sleep (ngủ) tiết kiệm pin. Nhược điểm là phạm vi hạn chế và Coordinator là single point of failure (điểm lỗi duy nhất).



Hình 1.2: Star topology – Coordinator ở trung tâm

b) Mesh Topology (Mạng lưới):

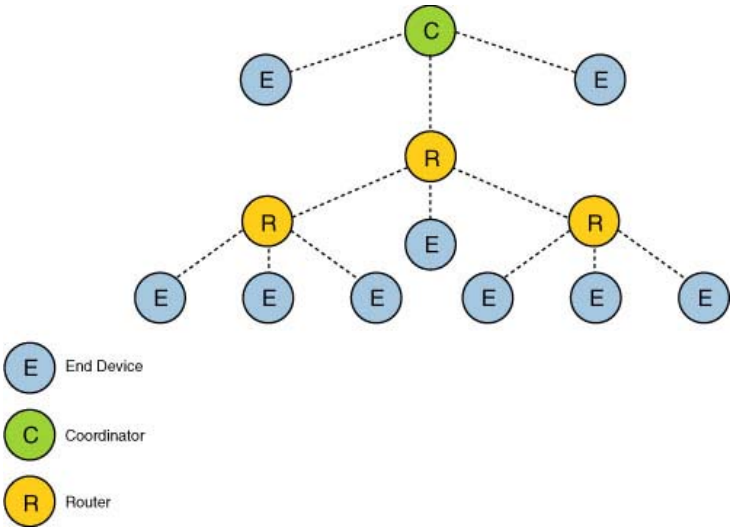
Mô hình mạnh mẽ nhất với Coordinator và nhiều Router tạo mạng lưới. Router có thể chuyển tiếp dữ liệu multi-hop (nhiều chặng), tự tìm đường thay thế khi có node lỗi. Phù hợp cho hệ thống lớn cần độ tin cậy cao.



Hình 1.3: Mesh topology – Định tuyến qua nhiều node

c) Tree Topology (Mạng hình cây):

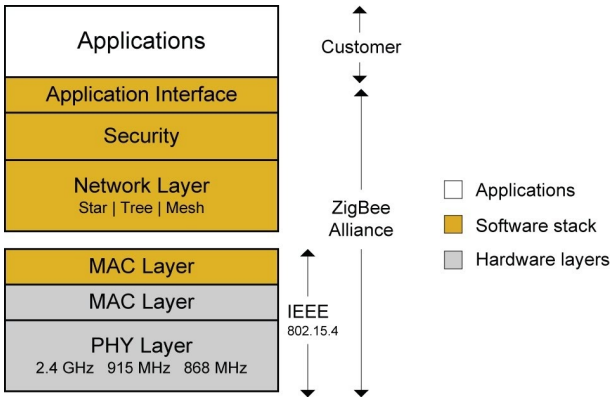
Cấu trúc phân cấp với Coordinator là gốc, Router làm nhánh, End Device là lá. Kết hợp ưu điểm của Star (đơn giản) và Mesh (mở rộng). Thường dùng trong các ứng dụng cần phân cấp quản lý rõ ràng.



Hình 1.4: Tree topology – Cấu trúc phân cấp

1.2.4 Kiến trúc Protocol Stack

Kiến trúc Zigbee Protocol Stack được xây dựng dựa trên mô hình phân tầng (layers), trong đó mỗi lớp cung cấp dịch vụ cụ thể cho lớp bên trên. Nó được chia thành hai phần chính: nền tảng IEEE 802.15.4 và các lớp do Zigbee Alliance định nghĩa.



Hình 1.5: Kiến trúc Zigbee Protocol Stack

1. Lớp Vật lý (PHY) và MAC (IEEE 802.15.4):

Đây là nền tảng phần cứng và truy cập kênh. PHY xử lý sóng vô tuyến (thường là 2.4 GHz) và điều chế tín hiệu. MAC quản lý việc truy cập kênh (CSMA-CA – Carrier Sense Multiple Access with Collision Avoidance), tạo liên kết cơ bản, và kiểm soát khung tin cậy (MAC ACK).

2. Lớp Mạng (NWK – Network Layer):

Do Zigbee định nghĩa. Lớp này chịu trách nhiệm **định tuyến (routing)** gói tin qua mạng lưới (mesh), quản lý địa chỉ mạng (16-bit), và xử lý bảo mật mạng (Network Key). Nó đảm bảo gói tin đi từ thiết bị nguồn đến đích, dù phải nhảy qua nhiều Router trung gian.

3. Lớp Ứng dụng (APL – Application Layer):

Đây là nơi các ứng dụng thực tế hoạt động, bao gồm 3 thành phần chính:

- **APS (Application Support Sub-layer):** Cầu nối giữa NWK và ứng dụng. Nó xử lý Binding (liên kết), phân mảnh tin nhắn (fragmentation) nếu gói tin quá lớn, và lọc các gói tin trùng lặp.

- **ZDO (Zigbee Device Object):** "Người quản lý" của thiết bị. Nó chịu trách nhiệm khởi tạo thiết bị, khám phá mạng, yêu cầu dịch vụ bảo mật, và quản lý các yêu cầu binding. ZDO luôn nằm ở **Endpoint 0**.
- **Application Framework:** Môi trường chứa các Application Objects (ứng dụng người dùng) nằm trên các Endpoint từ 1 đến 240. Tại đây, các thiết bị sử dụng ZCL (Zigbee Cluster Library) để định nghĩa hành vi (ví dụ: bật đèn, báo occupancy).

Security Service Provider (SSP): Bên cạnh các lớp trên là khối SSP, cung cấp dịch vụ mã hóa và xác thực cho cả lớp NWK và APS.

1.2.5 Địa chỉ và Bảo mật

Hệ thống địa chỉ:

- **IEEE Address (64-bit):** Địa chỉ vật lý duy nhất, cố định từ nhà máy
- **Short Address (16-bit):** Địa chỉ mạng được gán khi join (gia nhập mạng), dùng cho truyền dữ liệu
- **PAN ID (16-bit):** PAN – Personal Area Network (mạng cá nhân) ID là định danh mạng, phân biệt các mạng Zigbee khác nhau

Bảo mật:

- **Network Key:** Khóa 128-bit chia sẻ trong mạng (dùng chung cho tất cả thiết bị)
- **Link Key:** Khóa riêng giữa hai thiết bị (bảo mật từng cặp)
- **Trust Center:** Coordinator quản lý xác thực và phân phối khóa

1.2.6 Quá trình hình thành và gia nhập mạng

Giải thích thuật ngữ:

- **Form Network (Hình thành mạng):** Quá trình khởi tạo một mạng Zigbee mới. Chỉ thiết bị đóng vai trò Coordinator mới có thể thực hiện. Quá trình bao gồm quét năng lượng (Energy Scan) để chọn kênh vô tuyến ít nhiễu nhất và chọn một PAN ID duy nhất để không trùng với các mạng lân cận. Sau khi form xong, Coordinator tự gán cho mình địa chỉ `0x0000` và bắt đầu quản lý mạng.
- **Open-network (Permit Joining):** Hành động mở cửa mạng để cho phép thiết bị mới gia nhập. Mặc định mạng Zigbee sẽ đóng (không cho phép thiết bị lạ vào) để bảo mật. Khi lệnh này được kích hoạt (thường trong 180–254 giây), Coordinator (và các Router) sẽ phát đi tín hiệu trong bản tin Beacon rằng đang chấp nhận kết nối mới.
- **Network Steering:** Cơ chế (thường là plugin hoặc tính năng trong Zigbee 3.0 Base Device Behavior) giúp thiết bị tự động tìm và gia nhập mạng tốt nhất hiện có. Thay vì người dùng phải cấu hình thủ công kênh và PAN ID, thiết bị chạy network-steering sẽ tự động quét toàn bộ các kênh, tìm các mạng đang mở (Open-network), và thử gia nhập vào mạng có tín hiệu tốt nhất.

Chi tiết quy trình End Device (ED) tham gia mạng Coordinator:

Giả sử Coordinator đã Form mạng và đang Open-network. Quá trình ED gia nhập (Joining) diễn ra theo 4 giai đoạn:

Giai đoạn 1 – Khám phá mạng (Network Discovery):

1. ED bắt đầu quá trình Network Steering, gửi gói tin **Beacon Request** trên các kênh được hỗ trợ để tìm kiếm mạng.

2. Coordinator nhận được yêu cầu và gửi lại gói tin **Beacon** chứa cờ `Association Permit = TRUE` (do đang Open-network) và thông tin mạng (PAN ID, Extended PAN ID).

3. ED nhận Beacon, thấy mạng này đang mở và phù hợp, quyết định chọn Coordinator làm thiết bị cha (Parent).

Giai đoạn 2 – Kết nối MAC (MAC Association):

4. ED gửi gói tin **Association Request** chứa địa chỉ MAC 64-bit (EUI-64) của ED và thông tin năng lực (Capability Info) để báo rằng nó là thiết bị dùng pin hay nguồn lưới.

5. Coordinator gửi lại **MAC ACK** để xác nhận đã nhận yêu cầu, sau đó xử lý và gửi gói tin **Association Response** chứa địa chỉ mạng ngắn 16-bit (Short Address) mà Coordinator cấp phát cho ED (ví dụ: `0x1A2B`) và trạng thái `SUCCESS`.

Lưu ý: Lúc này ED đã kết nối về mặt vật lý nhưng chưa thể giao tiếp dữ liệu vì chưa có khóa bảo mật.

Giai đoạn 3 – Xác thực và Trao đổi khóa (Authentication & Key Transport):

6. Coordinator (đóng vai trò Trust Center) phát hiện có thiết bị mới. Nó lấy Network Key hiện tại của mạng, mã hóa bằng Link Key ban đầu. Với Zigbee 3.0, Link Key này có thể là khóa mặc định toàn cầu (`ZigbeeAlliance09`) hoặc khóa riêng từ Install Code nếu cấu hình bảo mật cao. Gói tin **Transport Key** (chứa Network Key đã mã hóa) được gửi xuống cho ED.

7. ED dùng Link Key ban đầu để giải mã, lấy được Network Key. Bây giờ ED đã có thể giải mã và gửi các gói tin dữ liệu trong mạng (trạng thái "Joined and Authenticated").

Giai đoạn 4 – Thông báo và Cập nhật bảo mật (Announcement & TCLK Update):

8. ED phát quảng bá gói tin **Device_annce** để thông báo cho toàn mạng biết: "Tôi là thiết bị mới với địa chỉ 16-bit này và địa chỉ 64-bit này".

9. Để đảm bảo an toàn (vì khóa mặc định ai cũng biết), ED phải yêu cầu đổi Link Key mới. ED gửi lệnh **Request Key** hoặc **Verify Key** tới Trust Center. Coordinator sinh ra Link Key mới (Trust Center Link Key – TCLK) ngẫu nhiên, mã hóa nó bằng Link Key cũ và gửi cho ED. Sau bước này, ED và Coordinator giao tiếp riêng tư bằng Link Key mới.

1.3 Zigbee Cluster Library (ZCL)

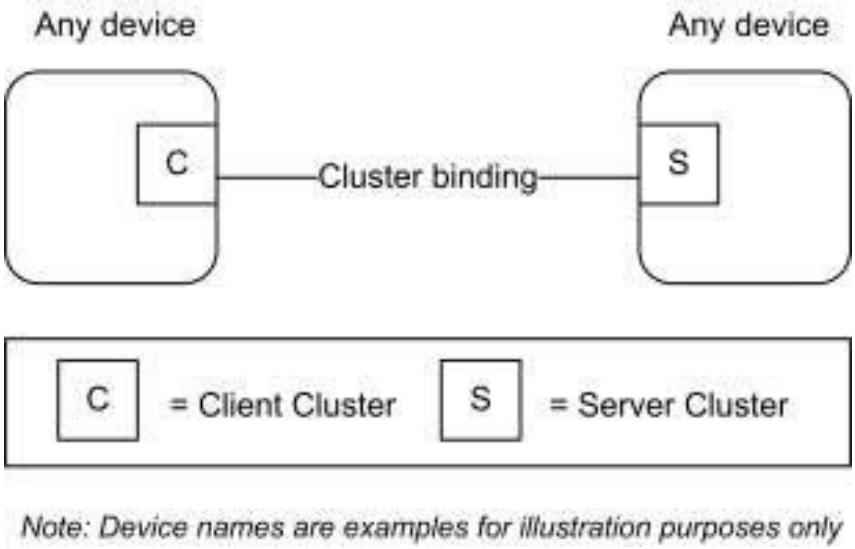
ZCL là một thư viện chuẩn hóa định nghĩa mọi chức năng mà một thiết bị Zigbee có thể thực hiện – giống như "cuốn từ điển ngôn ngữ" chung. Nó đảm bảo một bóng đèn của hãng A có thể hiểu lệnh "Bật" từ công tắc của hãng B.

1.3.1 Cấu trúc ZCL

- **Cluster (Cụm chức năng):** Là đơn vị cơ bản của ZCL. Mỗi Cluster đại diện cho một chức năng cụ thể (ví dụ: On/Off Cluster ID `0x0006` để bật/tắt, Level Control để chỉnh độ sáng, Occupancy Sensing Cluster ID `0x0406` để phát hiện hiện diện).
- **Attributes (Thuộc tính):** Các biến lưu trữ trạng thái (ví dụ: `OnOff = True/False`, `Occupancy = 0x01`).
- **Commands (Lệnh):** Các tin nhắn hành động (ví dụ: lệnh "Toggle", lệnh "Read Attribute").

Mô hình Client/Server: Mỗi Cluster chia làm hai phía:

- **Server (Máy chủ):** Nơi chứa dữ liệu thực tế (Attributes) và thực thi lệnh. Ví dụ: bóng đèn chứa On/Off Server để lưu trạng thái sáng/tối.
- **Client (Khách):** Nơi gửi lệnh điều khiển hoặc yêu cầu đọc dữ liệu. Ví dụ: công tắc chứa On/Off Client để gửi lệnh bật/tắt.



Hình 1.6: Mô hình Client/Server trong ZCL

1.3.2 Binding (Liên kết)

Binding là việc tạo ra một mối liên kết logic từ một Endpoint nguồn (thường là Client) tới một hoặc nhiều Endpoint đích (thường là Server) – giống như "đường dây nóng" giữa hai thiết bị.

Tự động hóa địa chỉ: Khi ứng dụng gửi lệnh qua một cluster đã được binding, nó không cần biết địa chỉ đích là gì. Zigbee Stack sẽ tự tra Bảng Binding (Binding Table) để điền địa chỉ đích vào gói tin.

Ứng dụng trong hệ thống Smart Home:

- **Báo cáo (Reporting):** Occupancy Sensor cần binding với Coordinator để tự động gửi báo cáo khi trạng thái thay đổi.
- **Điều khiển trực tiếp:** Switch binding trực tiếp với Light Device để điều khiển nhanh mà không cần qua Gateway xử lý.

1.3.3 Group (Nhóm)

Group là cơ chế gộp nhiều thiết bị vào một địa chỉ logic duy nhất (Group ID, 16-bit) để điều khiển đồng loạt – giống như "danh sách phát sóng".

Hiệu quả cao: Thay vì gửi 10 lệnh Unicast tới 10 bóng đèn để tắt chúng, chỉ cần gửi 1 lệnh Groupcast tới Group ID của phòng đó. Tất cả thiết bị trong nhóm sẽ nhận và xử lý lệnh cùng lúc.

- **Cluster Groups (0x0004):** Đây là một cluster đặc biệt dùng để quản lý việc thêm/bớt thiết bị vào các nhóm (lệnh Add Group, Remove Group).
- **Liên hệ với Binding:** Một entry trong Bảng Binding có thể trỏ tới một Group ID thay vì một thiết bị cụ thể (Multicast Binding). Khi đó, công tắc sẽ gửi lệnh tới cả nhóm.

1.4 Giao thức MQTT

1.4.1 Tổng quan

MQTT (Message Queuing Telemetry Transport) là giao thức nhắn tin nhẹ theo mô hình Publish/Subscribe (xuất bản/đăng ký), được thiết kế cho thiết bị IoT có tài nguyên hạn chế và mạng không ổn định. MQTT được phát triển bởi IBM vào năm 1999 và trở thành chuẩn OASIS năm 2014.

Tại sao cần MQTT thay vì HTTP?

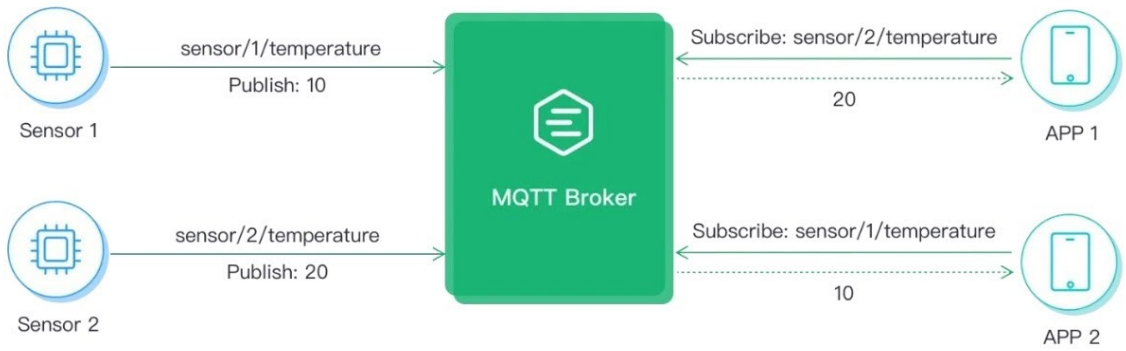
- **Nhẹ hơn:** Header MQTT tối thiểu 2 bytes, trong khi HTTP header có thể lên đến hàng trăm bytes.
- **Kết nối liên tục:** MQTT duy trì kết nối TCP, không cần handshake mỗi lần gửi như HTTP.
- **Hai chiều:** Server có thể chủ động đẩy dữ liệu xuống client (push), không cần client polling.
- **Phù hợp mạng kém:** Có cơ chế QoS và LWT để xử lý mất kết nối.

Mô hình Publish/Subscribe:

Thay vì giao tiếp trực tiếp (point-to-point), MQTT sử dụng mô hình gián tiếp qua Broker:

1. **Publisher** gửi tin nhắn lên một Topic (không cần biết ai nhận).
2. **Broker** nhận tin nhắn và phân phối đến tất cả Subscriber đã đăng ký Topic đó.
3. **Subscriber** nhận tin nhắn từ Broker (không cần biết ai gửi).

Mô hình này giúp tách rời hoàn toàn nguồn phát và nguồn nhận, dễ mở rộng và bảo trì.



Hình 1.7: Kiến trúc Publish/Subscribe của MQTT

1.4.2 Thành phần chính

- **Publisher:** Thiết bị gửi (xuất bản) tin nhắn lên Topic. Ví dụ: Gateway publish trạng thái đèn lên topic `sb/v1/.../devices/light/light_01/reported`.
- **Subscriber:** Thiết bị đăng ký nhận tin từ Topic. Ví dụ: Cloud Backend subscribe `sb/v1/.../#` để nhận tất cả dữ liệu từ gateway.
- **Broker:** Máy chủ trung gian định tuyến tin nhắn. Broker nhận tin từ Publisher, lọc theo Topic, và chuyển tiếp đến các Subscriber phù hợp.
- **Topic:** Chuỗi phân cấp dùng để định tuyến tin nhắn, phân tách bằng dấu "/". Ví dụ: `sb/v1/tenant1/site`
- **Client:** Bất kỳ thiết bị nào kết nối đến Broker. Một client có thể vừa là Publisher vừa là Subscriber.

1.4.3 Quality of Service (QoS)

QoS xác định mức độ đảm bảo tin nhắn đến được đích:

Bảng 1.1: Các mức QoS trong MQTT

Mức	Tên	Mô tả
QoS 0	At most once	Fire-and-forget: gửi một lần, không xác nhận. Có thể mất tin nhưng nhanh nhất. Phù hợp dữ liệu telemetry liên tục.
QoS 1	At least once	Đảm bảo đến đích ít nhất một lần. Gửi lại nếu không nhận ACK. Có thể trùng lặp. Phù hợp lệnh điều khiển.
QoS 2	Exactly once	Chính xác một lần, bắt tay 4 bước (PUBLISH, PUBREC, PUBREL, PUBCOMP). Chậm nhất nhưng không mất, không trùng.

Lưu ý: QoS áp dụng riêng cho từng chặng (Publisher→Broker và Broker→Subscriber). Mức QoS thực tế là min(QoS_publish, QoS_subscribe).

1.4.4 Retained Messages và Last Will

Retained Message (Tin nhắn lưu giữ):

- Khi publish với cờ Retain = true, Broker lưu tin nhắn cuối cùng của Topic.
- Subscriber mới sẽ nhận ngay tin nhắn này khi đăng ký, không cần đợi Publisher gửi lại.
- Ứng dụng: lưu trạng thái hiện tại (đèn đang bật/tắt, occupancy detected/clear).

Last Will and Testament (LWT):

- Client khai báo một "di chúc" khi kết nối (Will Topic, Will Message, Will QoS, Will Retain).
- Nếu Client ngắt kết nối đột ngột (mất mạng, crash), Broker tự động publish Will Message.
- Ứng dụng: báo trạng thái offline của gateway cho hệ thống giám sát.

1.4.5 Topics và Wildcards

Topics có cấu trúc phân cấp, phân tách bằng `/`. MQTT hỗ trợ wildcards khi Subscribe:

- `+` : Single-level wildcard – khớp đúng một cấp. VD: `sb/v1/+/+/gw1/gateway/health` khớp với nhiều tenant/site khác nhau.
- `#` : Multi-level wildcard – khớp từ 0 đến nhiều cấp, phải đặt cuối. VD: `sb/v1/tenant1/#` khớp với tất cả topic thuộc tenant1.

Quy ước đặt tên Topic (Best practices):

- Bắt đầu bằng namespace hệ thống và phiên bản: `sb/v1/...`
- Phân cấp logic: `sb/v1/{tenant}/{site}/{gateway}/{category}/...`
- Tránh bắt đầu bằng $ (dành cho system topics)
- Sử dụng phiên bản trong namespace để hỗ trợ migration

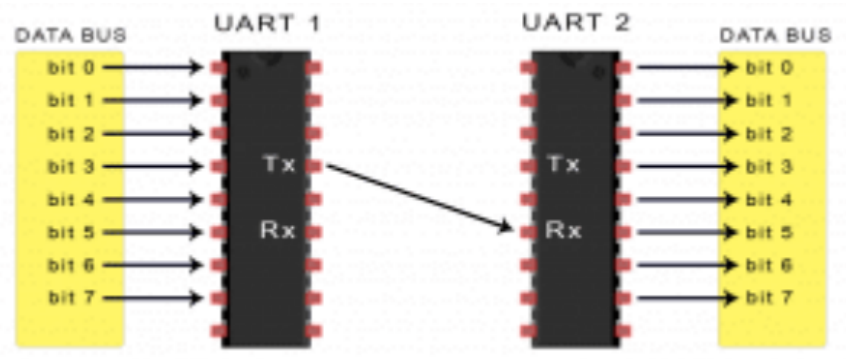
1.5 Giao tiếp UART

1.5.1 Tổng quan

UART (Universal Asynchronous Receiver/Transmitter) là giao thức truyền thông nối tiếp bất đồng bộ giữa hai thiết bị. Đây là một trong những giao thức cổ điển nhất và vẫn được sử dụng rộng rãi trong hệ thống nhúng do tính đơn giản và phổ biến.

Đặc điểm:

- **Hai dây chính:** TX (transmit - truyền) và RX (receive - nhận). TX của thiết bị A nối với RX của thiết bị B và ngược lại (crossed connection).
- **Full-duplex:** Có thể truyền và nhận đồng thời trên hai dây riêng biệt.
- **Point-to-point:** Thường chỉ kết nối hai thiết bị với nhau.
- **Không sử dụng clock chung:** Hai thiết bị phải thống nhất trước về tốc độ truyền (baud rate).



Hình 1.8: Kết nối UART giữa hai thiết bị

1.5.2 Frame Format

Mỗi UART frame gồm: Start bit (1), Data bits (5–9, thường 8), Parity bit (tùy chọn), Stop bit (1–2). Baud rate phổ biến: 9600, 115200.

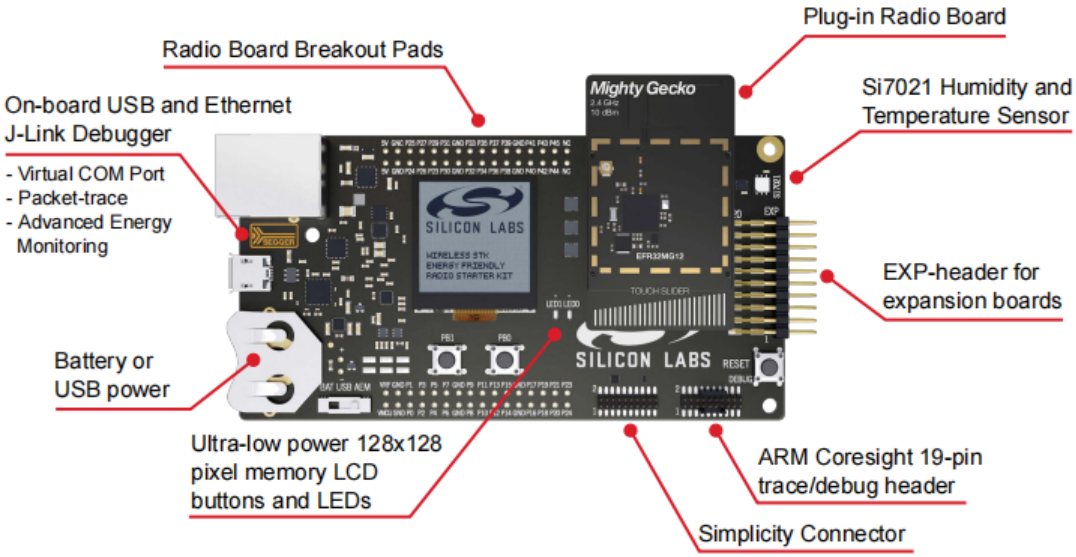
1.5.3 Ứng dụng UART trong dự án: EZSP/ASH

Trong dự án này, UART được sử dụng làm lớp vận chuyển vật lý (physical transport) giữa Z3Gateway host và NCP radio. Tuy nhiên, giao thức lớp ứng dụng trên đường UART không phải do dự án tự định nghĩa, mà sử dụng chuẩn EZSP/ASH của Silicon Labs:

- **EZSP** (EmberZNet Serial Protocol): Định nghĩa các command và callback giữa host và NCP. Host gửi EZSP command (ví dụ: "send ZCL On/Off command đến device X"), NCP thực thi và trả callback (ví dụ: "đã nhận attribute report từ device Y").
- **ASH** (Asynchronous Serial Host): Lớp framing bao bọc EZSP frame, cung cấp ACK/retry, CRC, flow control trên UART. ASH đảm bảo truyền tin cậy ngay cả khi có lỗi byte trên đường serial.

1.6 Phần cứng và Kit sử dụng

1.6.1 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit



Hình 1.9: EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit

Hệ thống sử dụng Silicon Labs Wireless Starter Kit làm nền tảng phần cứng phát triển và triển khai cho các node trong hệ thống Smart Home: Coordinator/NCP, Light Device, Switch Device và Occupancy Sensor. Mỗi node sử dụng một bộ kit riêng, gồm Radio Board BRD4162A và Mainboard BRD4001A.

a. **Radio Board BRD4162A**

BRD4162A là bo mạch vô tuyến dạng plug-in, đóng vai trò là khối xử lý trung tâm và thu phát không dây của node. Radio Board sử dụng SoC EFR32MG12 Wireless Gecko (mã EFR32MG12P332F1024GL125) hỗ trợ đa giao thức mạng lưới (mesh networking), với cấu hình bộ nhớ Flash 1024 kB và RAM 256 kB. Khối thu phát vô tuyến hoạt động ở băng tần 2.4 GHz, công suất phát tối đa 10 dBm, sử dụng anten tích hợp dạng Inverted-F PCB antenna.

Ngoài ra, BRD4162A tích hợp 8 Mbit Low-power Serial Flash (MX25R) phục vụ lưu trữ dữ liệu hoặc firmware cho cập nhật qua không khí (OTA), và tích hợp Capacitive Touch Slider 4 kênh.

b. **Mainboard BRD4001A**

Wireless Starter Kit Mainboard BRD4001A cung cấp nền tảng cấp nguồn, nạp/gỡ lỗi và ngoại vi tương tác cho quá trình phát triển. Mainboard tích hợp màn hình LCD Memory 128×128 pixel (Sharp LS013B7DH03), cảm biến Si7021 (I2C), 2 nút nhấn (BTN0, BTN1) và 2 LED (LED0, LED1).

c. **Nạp, gỡ lỗi và giám sát năng lượng**

BRD4001A tích hợp sẵn On-board Debugger (SEGGER J-Link), Advanced Energy Monitor (AEM), Packet Trace Interface (PTI) và Virtual COM Port (VCOM).

1.6.2 Cấu hình phần cứng theo thiết bị

a. **Coordinator / NCP Node**

- **Radio Board (BRD4162A):** MCU + NCP firmware (EZSP), Zigbee Coordinator stack.
- **USB J-Link (VCOM):** Virtual COM Port giao tiếp UART (EZSP/ASH) với Z3Gateway host.
- **PB0/PB1 Button:** Không sử dụng trực tiếp (điều khiển qua host).
- **LED0:** Network status indicator.
- **Power:** USB.

b. **Light Device (Router)**

- **Radio Board (BRD4162A):** MCU + Zigbee Router Stack + On/Off Server.
- **LED0:** Phản ánh trạng thái đèn: ON = sáng, OFF = tắt.
- **PB1 Button:** Network Steering (Join).
- **Power:** USB/DC (Router luôn bật).

c. **Switch Device (Router)**

- **Radio Board (BRD4162A):** MCU + Zigbee Router Stack + On/Off Client.
- **PB0 Button:** Toggle Light (gửi ZCL Toggle command).
- **PB1 Button:** Network Steering (Join).
- **Power:** USB/DC (Router luôn bật).

d. **Occupancy Sensor (Sleepy End Device)**

- **Radio Board (BRD4162A):** MCU + Zigbee SED Stack + Occupancy Sensing Server.
- **PB0 Button:** Mô phỏng phát hiện occupancy (hoặc sensor thực).
- **LED0:** Chỉ thị Awake/Sleep.
- **Power:** Battery (tiết kiệm năng lượng).

1.6.3 Toolchain và SDK

Bảng 1.2: Toolchain và SDK version

	Tên	**Version**
IDE	Simplicity Studio	5
SDK	Gecko SDK	4.5.0
Toolchain	GNU Arm	12.2.1
MCU	EFR32MG12P332F1024GL125	–
Radio Board	BRD4162A	–
Mainboard	BRD4001A	–

1.7 Luồng dữ liệu (Data Flow)

1.7.1 Luồng Uplink (State/Event Report)

Hình 1.10: Luồng dữ liệu Uplink

Luồng uplink mô tả đường đi của dữ liệu trạng thái và sự kiện từ thiết bị đầu cuối đến người dùng:

1. **End Device → NCP (Zigbee RF):** Occupancy Sensor phát hiện thay đổi trạng thái, đóng gói thành ZCL Report Attributes và gửi lên Coordinator/NCP qua mạng Zigbee. Tương tự, Light Device gửi state report khi trạng thái On/Off thay đổi.

2. **NCP → Z3Gateway Host (EZSP/ASH qua UART):** NCP radio nhận ZCL Report, chuyển lên host dưới dạng EZSP callback. Z3Gateway host xử lý callback, giải mã ZCL payload.

3. **Z3Gateway → MQTT Broker:** Module `telemetry_rx` chuẩn hóa dữ liệu thành JSON envelope và publish lên MQTT broker theo topic quy ước, ví dụ: `sb/v1/{tenant}/{site}/{gw}/de`

4. **MQTT Broker → Cloud Backend:** Cloud Backend (FastAPI) subscribe các topic reported/event, nhận dữ liệu và lưu vào database (bảng `device_states`, `events`).

5. **Cloud Backend → Mobile App:** Flutter App truy vấn trạng thái qua REST API hoặc subscribe trực tiếp MQTT để nhận cập nhật realtime.

1.7.2 Luồng Downlink (Command)

Hình 1.11: Luồng điều khiển Downlink

Luồng downlink mô tả đường đi của lệnh điều khiển từ người dùng đến thiết bị:

1. **Mobile App → Cloud Backend:** Người dùng nhấn nút điều khiển trên Flutter App. App gửi command request tới Cloud Backend qua REST API hoặc publish lên MQTT.

2. **Cloud Backend → MQTT Broker:** Cloud Backend tạo command message với `command_id` duy nhất và publish lên topic `sb/v1/.../commands/{cmd_id}/request`.

3. **MQTT Broker → Z3Gateway:** Z3Gateway subscribe topic command request, nhận payload chứa thông tin thiết bị đích và hành động cần thực hiện.

4. **Z3Gateway → NCP (EZSP):** Module `cmd_handler` chuyển đổi command thành EZSP request gửi xuống NCP, yêu cầu NCP phát ZCL On/Off Command qua Zigbee RF.

5. **NCP → Light Device (Zigbee RF):** NCP phát ZCL command xuống Light Device. Light thực thi lệnh, cập nhật LED, và gửi state report ngược lại qua luồng Uplink.

6. **Command Reply:** Z3Gateway publish kết quả thực thi lên topic `sb/v1/.../commands/{cmd_id}/r` để Cloud Backend và App biết trạng thái lệnh (thành công/thất bại/timeout).

1.7.3 Luồng Local Control

Ngoài luồng remote qua cloud, hệ thống hỗ trợ local control thông qua Zigbee Binding:

- Switch Device binding trực tiếp với Light Device qua On/Off Cluster.
- Khi người dùng nhấn nút trên Switch, ZCL Toggle Command được gửi trực tiếp đến Light mà không qua Gateway hay Cloud.
- Đảm bảo phản hồi nhanh (vài chục ms) và hoạt động ngay cả khi mất kết nối Internet.
- Gateway vẫn nhận state report từ Light sau khi trạng thái thay đổi, đảm bảo đồng bộ với Cloud.

1.8 MQTT Contract

Hệ thống sử dụng MQTT làm lớp contract chính giữa Gateway, Cloud Backend và Mobile App. Toàn bộ giao tiếp hệ thống cấp ứng dụng đi qua MQTT broker thay vì giao thức tùy chỉnh.

1.8.1 Namespace và Topic Structure

MQTT namespace được thiết kế phân cấp, hỗ trợ multi-tenant và multi-site:

```
sb/v1/{tenant_id}/{site_id}/{gateway_id}/...
```

Trong đó:

- `sb`: Smart Building – namespace gốc của hệ thống.
- `v1`: Phiên bản contract, cho phép migration khi cần thay đổi cấu trúc.
- `tenant_id`: Định danh khách hàng / tổ chức.
- `site_id`: Định danh site (tòa nhà / căn hộ).
- `gateway_id`: Định danh gateway trong site.

Bảng 1.3: Các nhóm MQTT topic chính

Topic Pattern	Hướng	Mô tả	QoS
`.../gateway/online`	Gw → All	Trạng thái online/offline (LWT)	1
`.../gateway/health`	Gw → Cloud	Thông tin sức khỏe gateway	0
`.../devices/{type}/{id}/reported`	Gw → Cloud	Trạng thái thực tế thiết bị	1
`.../devices/{type}/{id}/desired`	Cloud → Gw	Trạng thái mong muốn	1
`.../devices/{type}/{id}/telemetry`	Gw → Cloud	Dữ liệu telemetry định kỳ	0
`.../devices/{type}/{id}/event`	Gw → Cloud	Sự kiện (join, leave, error)	1
`.../commands/{cmd_id}/request`	Cloud → Gw	Yêu cầu thực thi lệnh	1
`.../commands/{cmd_id}/reply`	Gw → Cloud	Kết quả thực thi lệnh	1

1.8.2 Message Envelope

Tất cả message MQTT tuân theo cấu trúc envelope chung (JSON):

Mã nguồn 1.1: MQTT message envelope

```
{
  "schema": "sb.v1.device.reported",
  "msg_id": "uuid-unique",
  "ts": "2026-01-15T10:30:00Z",
  "tenant_id": "tenant1",
  "site_id": "site1",
  "gateway_id": "gw1",
  "source": "gateway",
  "payload": { ... }
}
```

Các trường envelope:

- `schema`: Định danh loại message, dùng để deserialize đúng kiểu.
- `msg_id`: UUID duy nhất cho mỗi message.
- `ts`: Timestamp ISO 8601.
- `source`: Nguồn phát message (`gateway`, `cloud`, `app`).
- `payload`: Dữ liệu thực tế, cấu trúc tùy thuộc schema.
- `trace_id` / `correlation_id`: (tùy chọn) dùng cho tracing và correlation giữa request–reply.

1.8.3 Retain và QoS Policy

- **Reported state:** QoS 1, Retain = true. Subscriber mới nhận ngay trạng thái hiện tại.
- **Telemetry:** QoS 0, Retain = false. Dữ liệu liên tục, mất một bản tin không ảnh hưởng lớn.
- **Command request/reply:** QoS 1, Retain = false. Đảm bảo lệnh được gửi đến nhưng không lưu giữ lệnh cũ.

- **Gateway online:** QoS 1, Retain = true, kết hợp LWT để tự động báo offline khi gateway mất kết nối.

CHƯƠNG 2
THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình thiết kế hệ thống Smart Home Platform, bao gồm: phân tích yêu cầu chức năng và phi chức năng, thiết kế vai trò thiết bị trong mạng Zigbee, thiết kế data model và MQTT message format, logic flow và command lifecycle cho các thành phần, kiến trúc cloud backend, và các quyết định trade-off.

2.1 Yêu cầu chức năng và phi chức năng

2.1.1 Yêu cầu chức năng

Hệ thống cần đáp ứng các yêu cầu chức năng sau:

- **Light Device:** Nhận lệnh bật/tắt từ Coordinator/NCP hoặc Switch (qua Binding), cập nhật trạng thái LED, gửi ZCL Report Attributes xác nhận trạng thái mới.
- **Switch Device:** Gửi lệnh On/Off/Toggle khi người dùng nhấn nút. Hỗ trợ local control qua Zigbee Binding (không cần gateway/cloud).
- **Occupancy Sensor:** Phát hiện sự hiện diện (occupied/unoccupied) và gửi ZCL Report Attributes lên Coordinator/NCP. Hỗ trợ chế độ Sleepy End Device tiết kiệm năng lượng.
- **Coordinator/NCP:** Hình thành và quản lý mạng Zigbee (form, permit join, trust center). Chạy NCP firmware, giao tiếp với host qua EZSP/ASH.
- **Z3Gateway Host:** Nhận EZSP callback từ NCP, chuẩn hóa thành MQTT message. Subscribe command topic, chuyển đổi thành EZSP request gửi xuống NCP. Theo dõi device registry và health.
- **Cloud Backend:** Lưu trữ device state, command history, event log. Cung cấp REST API cho mobile app. Tích hợp MQTT để nhận/gửi dữ liệu từ gateway.
- **Flutter App:** Hiển thị danh sách thiết bị và trạng thái realtime, điều khiển bật/tắt đèn, xem occupancy state, xem lịch sử sự kiện.

2.1.2 Yêu cầu phi chức năng

- **Thời gian phản hồi:** Thao tác điều khiển đèn từ app (từ lúc gửi command đến khi nhận reply) trong vài giây đối với luồng remote, dưới 100ms đối với local control qua Switch Binding.
- **Độ tin cậy:** Cơ chế MQTT QoS 1 cho command/state đảm bảo tin nhắn đến được đích ít nhất một lần. Command lifecycle tracking (accepted $\rightarrow$ sent $\rightarrow$ executed/failed/timeout) cho phép phát hiện và xử lý lỗi.

- **Tự phục hồi:** Gateway tự kết nối lại MQTT broker khi bị gián đoạn. NCP tự phục hồi mạng Zigbee khi mất nguồn tạm thời. Cloud backend retry khi mất kết nối MQTT.
- **State consistency:** Reported state được publish với Retain = true trên MQTT, đảm bảo subscriber mới luôn nhận trạng thái hiện tại. Cloud backend lưu state vào database, đảm bảo persistence.
- **Khả năng mở rộng:** Thiết kế MQTT namespace hỗ trợ multi-tenant, multi-site, multi-gateway. Database schema cho phép thêm device type mới mà không cần thay đổi cấu trúc.

2.2 Network và Device Roles (Zigbee)

Mỗi thiết bị trong mạng Zigbee được gán một vai trò cụ thể, quyết định khả năng routing, sleep, và tập hợp cluster hỗ trợ.

Bảng 2.1: Thiết kế Zigbee Device Roles

Device	Zigbee Role	Server Clusters	Client Clusters
Coordinator/NCP	ZC / Trust Center	–	On/Off Client
Light Device	ZR (Router)	On/Off (0x0006)	–
Switch Device	ZR (Router)	–	On/Off (0x0006)
Occupancy Sensor	ZED (SED)	Occupancy Sensing (0x0406)	–

Giải thích thiết kế vai trò:

- **Light Device là Router:** Light thường được cấp nguồn cố định (điện lưới), nên phù hợp chạy Router liên tục. Vai trò Router giúp mở rộng vùng phủ mạng Zigbee và tăng độ tin cậy routing.
- **Switch Device là Router:** Tương tự Light, Switch thường gắn cố định và có nguồn điện. Router role giúp Switch chuyển tiếp gói tin cho các node khác trong mạng.
- **Occupancy Sensor là Sleepy End Device (SED):** Sensor thường chạy pin, cần tiết kiệm năng lượng tối đa. SED cho phép node ngủ phần lớn thời gian, chỉ thức dậy khi có sự kiện hoặc theo chu kỳ poll.
- **Coordinator/NCP – Trust Center:** Chạy trên EFR32MG12 ở chế độ NCP, quản lý toàn bộ mạng, bảo mật, và phân phối khóa. Application logic được xử lý trên Z3Gateway host.

Binding design:

- Switch On/Off Client → Binding → Light On/Off Server: Cho phép local control trực tiếp không qua gateway.
- Occupancy Sensor Occupancy Sensing Server → Binding → Coordinator: Tự động gửi report khi trạng thái thay đổi.
- Light On/Off Server → Report Configuration → Coordinator: Báo cáo trạng thái thay đổi để gateway đồng bộ với cloud.

2.3 Data và Message Design

2.3.1 MQTT Message Channels

Hệ thống phân tách message thành các kênh (channel) theo chức năng. Mỗi kênh có ngữ nghĩa và QoS/retain policy riêng:

Bảng 2.2: Thiết kế các kênh MQTT

Kênh	Mô tả	QoS	Retain	Ví dụ dữ liệu
reported	Trạng thái thực tế do gateway báo cáo	1	true	`{"on_off": true}`
desired	Trạng thái mong muốn do cloud yêu cầu	1	true	`{"on_off": false}`
telemetry	Dữ liệu đo đạc định kỳ	0	false	`{"rssi": -45}`
event	Sự kiện hệ thống	1	false	`{"type": "joined"}`
request	Command request từ cloud	1	false	`{"action": "on"}`
reply	Command reply từ gateway	1	false	`{"status": "ok"}`

2.3.2 State Message Design

Reported state message cho Light Device:

Mã nguồn 2.1: Light reported state message

```
{
  "schema": "sb.v1.device.reported",
  "msg_id": "550e8400-e29b-41d4-a716",
  "ts": "2026-01-15T10:30:00Z",
  "source": "gateway",
  "payload": {
    "device_id": "light_01",
    "device_type": "light",
    "eui64": "00:11:22:33:44:55:66:77",
    "nwk_addr": "0x1A2B",
    "on_off": true,
    "endpoint": 1
  }
}
```

Occupancy event message:

Mã nguồn 2.2: Occupancy event message

```
{
  "schema": "sb.v1.device.reported",
  "msg_id": "661f9500-f30c-52e5-b827",
```

```
  "ts": "2026-01-15T10:30:05Z",
  "source": "gateway",
  "payload": {
    "device_id": "occ_sensor_01",
    "device_type": "occupancy_sensor",
    "eui64": "AA:BB:CC:DD:EE:FF:00:11",
    "occupancy": 1,
    "occupancy_sensor_type": "pir"
  }
}
```

2.3.3 Command Message Design

Command request (Cloud → Gateway):

Mã nguồn 2.3: Command request message

```
{
  "schema": "sb.v1.command.request",
  "msg_id": "cmd-uuid-001",
  "ts": "2026-01-15T10:31:00Z",
  "source": "cloud",
  "payload": {
    "command_id": "cmd-uuid-001",
    "target_device": "light_01",
    "target_type": "light",
    "action": "set_on_off",
    "params": {
      "on_off": true
    }
  }
}
```

Command reply (Gateway → Cloud):

Mã nguồn 2.4: Command reply message

```
{
  "schema": "sb.v1.command.reply",
  "msg_id": "reply-uuid-001",
  "ts": "2026-01-15T10:31:02Z",
  "source": "gateway",
  "correlation_id": "cmd-uuid-001",
  "payload": {
    "command_id": "cmd-uuid-001",
    "status": "executed",
    "message": "ZCL On/Off command sent successfully"
  }
}
```

2.3.4 Versioning Rule

Hệ thống áp dụng nguyên tắc versioning cho message contract:

- **Namespace versioning:** Phiên bản nằm trong MQTT topic namespace (`sb/v1/...`). Khi cần breaking change, tạo namespace mới (`sb/v2/...`) và chạy song song trong giai đoạn migration.
- **Schema field:** Mỗi message chứa trường `schema` (ví dụ: `sb.v1.device.reported`) cho phép consumer phân biệt và deserialize đúng kiểu.
- **Backward compatibility:** Thêm field mới vào payload không phá vỡ consumer cũ (consumer bỏ qua field không biết). Xóa field hoặc đổi kiểu dữ liệu yêu cầu nâng phiên bản.

2.4 Logic Flow và State Machine

2.4.1 Light Device State Machine

Light Device có state machine đơn giản với 2 trạng thái chính:

- **OFF** (trạng thái mặc định sau khi join mạng)
- **ON**

Transition conditions:

- OFF → ON: Nhận ZCL On Command hoặc Toggle Command (khi đang OFF).
- ON → OFF: Nhận ZCL Off Command hoặc Toggle Command (khi đang ON).
- Nhận lệnh từ: Coordinator/NCP (remote command qua gateway), Switch Device (local binding), hoặc Groupcast command.

Sau mỗi transition, Light Device:

1. Cập nhật thuộc tính `OnOff` trong On/Off Server Cluster.
2. Đổi trạng thái LED vật lý.
3. Gửi ZCL Default Response (nếu yêu cầu) và/hoặc Report Attributes lên Coordinator.

2.4.2 Command Lifecycle

Mỗi command trong hệ thống đi qua một vòng đời (lifecycle) với các trạng thái theo dõi rõ ràng:

1. **accepted:** Cloud Backend nhận command request từ app, gán `command_id`, lưu vào database với trạng thái "accepted", publish lên MQTT.
2. **queued:** Gateway nhận command từ MQTT, xếp vào hàng đợi xử lý nội bộ.
3. **sent:** Gateway đã chuyển đổi command thành EZSP request và gửi xuống NCP. NCP phát ZCL command qua Zigbee RF.
4. **executed:** Gateway nhận xác nhận (ZCL Default Response hoặc state report) rằng thiết bị đã thực thi thành công. Publish reply với `status = "executed"`.
5. **failed:** ZCL command bị từ chối hoặc thiết bị không phản hồi đúng. Publish reply với `status = "failed"` kèm lý do.

6. **timeout:** Không nhận được phản hồi trong khoảng thời gian quy định. Publish reply với `status = "timeout"`.

Cloud Backend cập nhật trạng thái command trong database khi nhận reply, cho phép app hiển thị tiến trình và kết quả.

2.4.3 Gateway Runtime Flow

Z3Gateway host hoạt động theo event-driven loop trong single process:

1. **Khởi động:**
   - Mở kết nối UART tới NCP, khởi tạo EZSP/ASH.
   - Kết nối MQTT broker, subscribe command topics.
   - Khôi phục trạng thái mạng Zigbee từ NCP (nếu đã form trước đó).
2. **Main loop:**
   - Poll EZSP events từ NCP: xử lý device joined/left, attribute report, ZCL response.
   - Poll MQTT messages: xử lý command request từ cloud.
   - Module `device_monitor`: cập nhật registry, kiểm tra health.
   - Module `telemetry_rx`: chuẩn hóa ZCL data, publish lên MQTT.
   - Module `cmd_handler`: xử lý command queue, gửi EZSP request, theo dõi timeout.
3. **Xử lý lỗi:**
   - MQTT disconnect: tự động reconnect với exponential backoff.
   - UART error: reset EZSP/ASH connection.
   - Publish gateway health status định kỳ.

2.5 Cloud Backend Architecture

2.5.1 Kiến trúc tổng quan

Cloud Backend được xây dựng với FastAPI framework, cung cấp hai vai trò chính:

1. **MQTT Integration:** Subscribe các topic từ gateway để nhận state/event, publish command request. Hoạt động như MQTT consumer/producer chạy nền.
2. **REST API:** Cung cấp HTTP endpoints cho mobile app và dashboard. Truy vấn trạng thái thiết bị, gửi command, xem event history.

2.5.2 Database Design

Database sử dụng SQLAlchemy ORM với SQLite (giai đoạn phát triển), hướng tới PostgreSQL cho production. Các bảng chính:

Bảng 2.3: Các bảng database chính

Bảng	Mô tả
`devices`	Danh sách thiết bị đã đăng ký: device_id, device_type, eui64, nwk_addr, name, gateway_id, created_at, updated_at.
`device_states`	Trạng thái hiện tại: device_id, state_json (on_off, occupancy...), last_reported_at.
`events`	Lịch sử sự kiện: event_id, device_id, event_type, payload_json, timestamp.
`commands`	Lệnh điều khiển: command_id, target_device, action, params_json, status, created_at, updated_at.

2.5.3 REST API Endpoints

Bảng 2.4: REST API endpoints chính

Method	Endpoint	Mô tả
GET	`/health`	Health check
GET	`/devices`	Danh sách thiết bị
GET	`/devices/{id}`	Chi tiết và trạng thái thiết bị
POST	`/commands`	Gửi command điều khiển
GET	`/commands/{id}`	Trạng thái command
GET	`/events`	Lịch sử sự kiện (phân trang)

2.5.4 MQTT – Database Integration

Cloud Backend chạy một MQTT subscriber loop chạy nền, xử lý message từ gateway:

- **Nhận reported state:** Parse message, cập nhật bảng `device_states` với trạng thái mới. Nếu device chưa tồn tại, tự động tạo entry trong bảng `devices`.
- **Nhận event:** Lưu vào bảng `events` để truy vấn lịch sử.
- **Nhận command reply:** Cập nhật trạng thái command trong bảng `commands` (executed/failed/timeout)
- **Gửi command:** Khi REST API nhận POST `/commands`, Backend tạo record trong database, publish message lên MQTT command request topic.

2.6 Trade-offs và Quyết định thiết kế

- **Telemetry interval vs traffic:** Tần suất report cao giúp cloud/app luôn có dữ liệu mới nhất, nhưng tăng lưu lượng mạng Zigbee và MQTT. Thiết kế hiện tại dùng event-driven reporting (chỉ báo khi có thay đổi) thay vì periodic polling, giảm đáng kể traffic mà vẫn đảm bảo tính thời gian thực.
- **QoS choices:** QoS 0 cho telemetry (chấp nhận mất vài bản tin, ưu tiên throughput), QoS 1 cho command và state (đảm bảo đến đích, chấp nhận duplicate xử lý bằng idempotency). QoS 2 không sử dụng vì overhead cao không cần thiết cho use case hiện tại.

- **Local control vs Cloud control:** Switch → Light direct binding cho phản hồi nhanh (<100ms) và hoạt động offline. Cloud → Gateway → Light cho phép điều khiển từ xa và logging/analytics. Hai luồng bổ trợ nhau: local cho UX tốt, cloud cho trải nghiệm đầy đủ.
- **NCP model vs SoC model cho Coordinator:** NCP tách Zigbee stack (radio chip) và application logic (host), cho phép gateway có tài nguyên dồi dào (RAM, CPU, network) để chạy MQTT, database, rules engine. SoC phù hợp hơn cho end device đơn giản (light, switch). Dự án chọn NCP cho coordinator/gateway và SoC cho end device.
- **Single-process vs Multi-process Gateway:** Kiến trúc hiện tại tích hợp MQTT trực tiếp trong Z3Gateway C process (single-process). Ưu điểm: đơn giản, ít latency, dễ deploy. Nhược điểm: khó mở rộng module độc lập. Hướng phát triển: có thể tách thành gateway adapter + MQTT bridge qua IPC (NDJSON socket) khi cần modularity.
- **FastAPI + SQL vs Firebase:** FastAPI + SQLAlchemy cho phép kiểm soát hoàn toàn schema, dễ tùy biến logic, chi phí thấp (self-hosted). Firebase đơn giản hơn cho prototype nhưng hạn chế truy vấn phức tạp và phụ thuộc vendor. Dự án chọn FastAPI để có flexibility cho long-term development.

CHƯƠNG 3
TRIỂN KHAI HỆ THỐNG

Chương này trình bày quá trình triển khai hệ thống Smart Home Platform, bao gồm: firmware cho các node Zigbee (Coordinator, Light, Switch, Occupancy), Gateway Service, Cloud Firebase integration, Flutter Mobile App, và luồng end-to-end command/state flow.

3.1 Triển khai Firmware

Vai trò các node Zigbee và cơ chế ZCL đã được trình bày ở Chương 1. Phần này mô tả cách thiết kế đó được ánh xạ vào các module firmware cụ thể.

3.1.1 Cấu trúc Firmware Coordinator

Coordinator là node trung tâm của mạng Zigbee, chịu trách nhiệm tạo network, quản lý thiết bị join/leave, và bridge dữ liệu với Gateway qua UART.

Bảng 3.1: Coordinator Firmware – Ánh xạ module

Module	**Chức năng chi tiết**	**File**
`net_mgr`	Quản lý network: form network khi khởi động, permit join (PB1, 180s), device table.	`net_mgr.c`
`uart_link`	Giao tiếp UART với Gateway: parse @CMD, format @DATA/@ACK/@INFO. Interrupt-driven RX.	`uart_link.c`
`state_rx`	Nhận ZCL Report Attributes từ Light Device và Occupancy Sensor, cập nhật app_state, trigger @DATA.	`state_rx.c`
`cmd_handler`	Parse command từ @CMD, validate target device, gửi ZCL On/Off command xuống Light Device.	`cmd_handler.c`
`app_state`	Lưu trữ trạng thái runtime: device list, light state, occupancy state, timestamps.	`app_state.c`
`lcd_ui`	Điều khiển LCD 128×128: hiển thị network status, device list, light/occupancy state.	`lcd_ui.c`

Cấu hình Zigbee Network:

- **Channel**: Mặc định channel 11 (2.405 GHz).
- **PAN ID**: Auto-generated ngẫu nhiên khi form network.
- **Permit Join Duration**: 180 giây khi nhấn PB1.
- **Security Mode**: Centralized Trust Center.

3.1.2 Firmware Light Device

Light Device đóng vai trò Router, luôn bật để nhận command bất kỳ lúc nào và hỗ trợ routing cho các node khác.

Bảng 3.2: Light Device Firmware – Cấu trúc

Module	Chức năng chi tiết	File
`app`	Nhận ZCL On/Off Command, cập nhật LED0 tương ứng, gửi ZCL Default Response / Report Attributes xác nhận trạng thái.	`app.c`
`main`	Khởi tạo hệ thống, cấu hình Zigbee Router Stack, chạy vòng lặp sự kiện.	`main.c`

Indicator LED: LED0 phản ánh trạng thái đèn: **ON** = LED sáng, **OFF** = LED tắt.

3.1.3 Firmware Switch Device

Switch Device đóng vai trò Router, chứa On/Off Client Cluster. Khi người dùng nhấn nút, Switch gửi ZCL Toggle Command tới Light Device qua Binding.

Bảng 3.3: Switch Device Firmware – Cấu trúc

Module	Chức năng chi tiết	File
`app`	Xử lý sự kiện nút nhấn (PB0): gửi ZCL On/Off Toggle command tới Light Device qua Binding Table.	`app.c`
`main`	Khởi tạo, cấu hình Zigbee Router Stack, Network Steering khi nhấn PB1.	`main.c`

3.1.4 Firmware Occupancy Sensor

Occupancy Sensor đóng vai trò Sleepy End Device (SED), tối ưu cho hoạt động bằng pin.

Bảng 3.4: Occupancy Sensor Firmware – Cấu trúc

Module	Chức năng chi tiết	File
`app`	Đọc trạng thái occupancy (button mô phỏng hoặc sensor thực), gửi ZCL Report Attributes (Occupancy Sensing Cluster) lên Coordinator theo event hoặc interval.	`app.c`
`main`	Khởi tạo, cấu hình Zigbee SED Stack, sleep mode EM2.	`main.c`

Tham số runtime:

- **Report mode**: Event-driven (khi occupancy thay đổi) + periodic heartbeat.
- **Sleep mode**: EM2 – RAM retained, RTC running.
- **Parent poll interval**: 1 giây.

3.2 Triển khai Gateway Service

Gateway Service chạy trên Linux (Ubuntu), đóng vai trò bridge giữa mạng Zigbee (qua UART) và Cloud (qua MQTT).

3.2.1 Thiết lập môi trường

Bảng 3.5: Công cụ và phiên bản sử dụng

Thành phần	Công cụ	Phiên bản
Firmware IDE	Simplicity Studio	v5
SDK	Gecko SDK	4.5.x
MQTT Broker	Mosquitto (local) / Cloud	$\geq$2.0
Gateway	Python	$\geq$3.10
Cloud	Firebase	(latest)
Mobile App	Flutter	$\geq$3.x

3.2.2 Kiến trúc đa luồng

Gateway cần xử lý đồng thời nhiều nguồn sự kiện: dữ liệu từ UART, bản tin MQTT (lệnh điều khiển), và cơ chế chờ ACK/timeout. Gateway được tổ chức theo kiến trúc đa luồng:

- **Luồng chính (Main Thread):** Khởi tạo, apply rules, build @CMD, retry/timeout, publish MQTT.
- **Luồng đọc UART (UART Reader Thread):** Readline, parse @INFO/@DATA/@ACK, cập nhật shared state.
- **Vòng lặp MQTT (MQTT Network Loop):** Connect, subscribe, on_message callback, enqueue command.

3.2.3 Triển khai UART Line Protocol

Giao thức UART text line đã được đặc tả ở Mục **??**. Grammar triển khai:

Mã nguồn 3.1: Định dạng khung UART

```
FRAME := PREFIX SPACE JSON CRLF
PREFIX := "@INFO" | "@DATA" | "@ACK" | "@CMD" | "@LOG"
SPACE := " " (single space)
JSON := valid JSON object (no embedded newlines)
CRLF := "\r\n"

# Example frames:
@DATA {"device":"light_01","state":"on","ts":1706284800}
@DATA {"device":"occupancy_01","occupied":true,"ts":1706284801}
@CMD {"cid":"abc123","op":"light_set","target":"light_01","value":"on"}
@ACK {"cid":"abc123","ok":true,"msg":"light set to on"}
@INFO {"version":"2.0.0","uptime":3600,"devices":4}
```

3.2.4 Cơ chế Correlation ID

Khi App gửi lệnh điều khiển đèn, Gateway cần biết ACK nhận được thuộc về lệnh nào. Giải pháp: **Correlation ID (cid)** – mã định danh duy nhất gắn vào mỗi lệnh.

Mã nguồn 3.2: Cơ chế đối sánh CID và ID lệnh

```
# gateway/service.py - Simplified, core logic only
class AckRouter:
    def __init__(self):
        self._pending = {} # cid -> {event, result}
        self._lock = Lock()

    def register(self, cid: str):
        self._pending[cid] = {"event": Event(), "result": None}

    def wait_for_ack(self, cid: str, timeout_s: float):
        entry = self._pending.get(cid)
        if entry and entry["event"].wait(timeout=timeout_s):
            return self._pending.pop(cid)["result"]
        return None

    def resolve(self, cid: str, payload: dict):
        if cid in self._pending:
            self._pending[cid]["result"] = payload
            self._pending[cid]["event"].set()
```

3.2.5 Chính sách Timeout và Retry

Gateway triển khai retry với exponential backoff:

Bảng 3.6: Tham số Timeout và Retry

Tham số	Giá trị	Ý nghĩa
`ack_timeout_s`	3	Thời gian chờ ACK tối đa cho mỗi lần thử.
`max_retries`	2	Số lần retry tối đa (tổng 3 lần thử).
`cmd_retry_base_delay_s`	0.3	Base delay cho exponential backoff.
`cmd_retry_max_delay_s`	1.2	Delay tối đa (cap).
`cmd_retry_jitter_s`	0.2	Random jitter.

Công thức:

$$delay = \min(max_delay, base_delay \times 2^{attempt-1}) + random(0, jitter) \tag{3.1}$$

3.2.6 Rules Engine

Rules Engine kiểm soát quyền gửi command:

1. **Global Lock**: Cờ maintenance mode. Nếu true, reject tất cả command.
2. **CID Deduplication**: Tránh lặp lại command.

3. **User Cooldown**: Thời gian tối thiểu giữa 2 command của cùng user.
4. **Global Cooldown**: Thời gian tối thiểu giữa 2 command bất kỳ.

3.2.7 Admin API

Admin API cung cấp HTTP endpoints cho giám sát và cấu hình Gateway từ xa. Mặc định bind trên `127.0.0.1:8080`.

Bảng 3.7: Admin API Endpoints

Method	Endpoint	Mô tả
GET	`/health`	Health check: UART, MQTT, uptime
GET	`/config`	Lấy config hiện tại
POST	`/config`	Cập nhật config runtime
GET	`/rules`	Lấy rules hiện tại
POST	`/rules`	Cập nhật rules (lock, cooldown)
GET	`/logs`	Lấy N dòng log gần nhất

3.3 Triển khai Cloud (Firebase)

3.3.1 Firebase Setup

(Viết chi tiết: Firebase project setup, Realtime DB / Firestore configuration, security rules, Cloud Functions nếu có.)

3.3.2 Device State Sync

(Viết cách Gateway publish state lên MQTT, và cách Firebase subscribe/store state. Hoặc gateway push trực tiếp qua Firebase REST API.)

3.3.3 Event History và Logging

(Viết cách lưu event log: command history, state changes, occupancy events.)

3.4 Triển khai Flutter Mobile App

3.4.1 App Architecture

(Viết kiến trúc Flutter app: state management, Firebase integration, UI layer.)

3.4.2 Device Control UI

(Viết UI cho điều khiển bật/tắt đèn, hiển thị trạng thái.)

3.4.3 Realtime State Display

(Viết cơ chế subscribe Firebase Realtime DB để cập nhật UI.)

3.4.4 Occupancy Monitoring

(Viết hiển thị trạng thái occupancy, history.)

3.5 Luồng End-to-end Command/State

3.5.1 App → Cloud → Gateway → Light

(Viết chi tiết sequence: Flutter app gửi command → Firebase → Gateway subscribe → UART @CMD → Coordinator → ZCL On/Off → Light Device → state report quay lại.)

3.5.2 Local Automation: Occupancy → Light

(Viết hướng triển khai local automation: Occupancy Sensor phát hiện → Coordinator logic → tự động bật/tắt Light Device mà không cần qua Cloud.)

3.6 Hướng phát triển OTA

(Viết tổng quan hướng OTA: Gecko Bootloader, GBL file, OTA qua Zigbee, OTA campaign management ở mức overview/direction.)

CHƯƠNG 4
KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này trình bày kết quả chạy hệ thống Smart Home Platform, bao gồm điều kiện thử nghiệm, kết quả end-to-end, kết quả từng thành phần và đánh giá hiệu năng.

4.1 Điều kiện chạy thử (Test Setup)

Hardware:

- 1x Coordinator: EFR32MG12 Wireless Starter Kit (BRD4162A)
- 1x Light Device: EFR32MG12 Radio Board (BRD4162A)
- 1x Switch Device: EFR32MG12 Radio Board (BRD4162A)
- 1x Occupancy Sensor: EFR32MG12 Radio Board (BRD4162A)
- Gateway: Ubuntu Linux (laptop / Raspberry Pi)

Software:

- Simplicity Studio v5 with Gecko SDK 4.5.x
- MQTT Broker (Mosquitto / Cloud)
- Firebase (Realtime DB / Firestore)
- Flutter App (Android / iOS)

Connectivity:

- UART: 115200 baud, 8N1
- MQTT: Cloud broker, QoS 0/1
- Zigbee: Channel 11, centralized Trust Center

4.2 Kết quả End-to-end

(Viết kết quả kiểm chứng 2 luồng chính:)

- **Luồng Uplink:** Occupancy Sensor / Light → Zigbee → Coordinator → UART → Gateway → MQTT → Cloud → Flutter App
- **Luồng Downlink:** Flutter App → Cloud → MQTT → Gateway → UART → Coordinator → Zigbee → Light Device → ACK/State → App

4.3 Kết quả từng thành phần

4.3.1 Zigbee Network

(Viết kết quả: form network, join, binding, command/report hoạt động. Thêm hình khi có.)

4.3.2 Gateway Service

(Viết kết quả: UART parse, MQTT bridge, rules engine, ACK routing.)

4.3.3 Cloud Firebase

(Viết kết quả: device state sync, command relay, event logging.)

4.3.4 Flutter Mobile App

(Viết kết quả: UI screenshots, realtime update, light control.)

4.4 Đánh giá hiệu năng

(Viết đánh giá: latency end-to-end, reliability, energy profiler kết quả nếu có.)

CHƯƠNG 5
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết mức độ hoàn thành các mục tiêu đề ra, đánh giá đóng góp của đồ án, phân tích các hạn chế và đề xuất hướng phát triển tiếp theo.

5.1 Kết luận

(Viết tổng kết thành quả đạt được cho dự án smart home:)

- **Xây dựng mạng Zigbee hoạt động ổn định:** (Viết: mạng Zigbee 3.0 với 4 device types, firmware module hóa.)
- **Thiết kế giao thức truyền thông UART-MQTT:** (Viết: UART bridge, MQTT topics, CID/ACK/Retry, Rules Engine, LWT.)
- **Tích hợp Cloud Firebase:** (Viết: device state sync, command relay, event history.)
- **Phát triển Flutter Mobile App:** (Viết: light control, occupancy monitoring, realtime updates.)
- **Kiến trúc IoT end-to-end có thể tái sử dụng:** (Viết: kiến trúc từ firmware đến app, tách biệt rõ ràng, dễ mở rộng.)

5.2 Hạn chế

- **Quy mô thử nghiệm nhỏ:** Kết quả được thu thập với số node hạn chế trong phạm vi phòng. Chưa có dữ liệu về hiệu năng khi mở rộng lên nhiều node hoặc môi trường có nhiều vật cản.
- **Bảo mật chưa đầy đủ:** Hệ thống hiện tại chỉ có bảo mật mạng Zigbee. Giao tiếp MQTT có thể chưa mã hóa TLS đầy đủ. Flutter App cần bổ sung xác thực mạnh hơn.
- **Chưa có lưu trữ dữ liệu dài hạn chuyên dụng:** Event history lưu trong Firebase nhưng chưa có chính sách retention và analytics chuyên sâu.
- **Điểm lỗi đơn:** Coordinator là điểm duy nhất kết nối mạng Zigbee với Gateway. Nếu Coordinator hoặc Gateway gặp sự cố, hệ thống ngừng hoạt động.
- **Local automation hạn chế:** Chưa có rule engine phía Coordinator/Gateway cho automation phức tạp (ví dụ: occupancy → bật đèn tự động với điều kiện thời gian).

5.3 Hướng phát triển

- **Chuẩn hóa kiến trúc Gateway theo hướng Z3Gateway / NCP-host:** Phân tách rõ Coordinator (quản lý mạng Zigbee) và Gateway host (chuyển tiếp, logic, API). Migrate từ SoC Coordinator sang NCP model để Gateway có khả năng xử lý mạnh hơn.
- **Migrate Gateway sang Raspberry Pi:** Chuyển Gateway service từ Ubuntu laptop sang Raspberry Pi để tiến tới embedded gateway triển khai thực tế.
- **Tăng cường bảo mật:** MQTT TLS (port 8883), xác thực mạnh cho Firebase/App, phân quyền theo vai trò, audit log.
- **OTA (Over-The-Air firmware update):** Triển khai Gecko Bootloader + GBL file cho Zigbee OTA. Quản lý OTA campaign từ Cloud.
- **Local automation engine:** Xây dựng rule engine tại Gateway/Coordinator cho automation không phụ thuộc cloud (occupancy $\rightarrow$ light, schedule-based control).
- **Cảnh báo và thông báo:** Push notification qua Firebase Cloud Messaging khi có sự kiện: occupancy detected, device offline, gateway disconnect.
- **Mô hình nhiều nhà (multi-home):** Mỗi nhà một Gateway; Cloud Firebase quản lý tập trung nhiều home. User có thể quản lý nhiều nhà từ một app.
- **Mở rộng device types:** Thêm dimmer (Level Control Cluster), temperature sensor, door/window sensor, smart plug.

TÀI LIỆU THAM KHẢO

[1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.

[2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.

[3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.

[4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.

[5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.

[6] Gryglecki, M. (HiveMQ). (n.d.). *Speaker Deck: MQTT Broker 100 - Basic*.

[7] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.

[8] Circuit Basics. (n.d.). *Basics of UART Communication*.

[9] Texas Instruments. (2010). *KeyStone Architecture Universal Asynchronous Receiver/Transmitter (UART) User Guide*. Literature Number: SPRUGP1.

[10] Silicon Labs. (n.d.). *UG261: EFR32MG12 2.4 GHz 10 dBm Radio Board User's Guide*. (Dành cho bo mạch BRD4162A và các kit liên quan).

[11] Silicon Labs. (n.d.). *UG489: Silicon Labs Gecko Bootloader User's Guide for GSDK 4.0 and Higher*.

[12] Silicon Labs. (n.d.). *Gecko Platform Release Notes 4.4.5.0*.

[13] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.

[14] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.

[15] Google. (n.d.). *Firebase Documentation*. https://firebase.google.com/docs.

[16] Google. (n.d.). *Firebase Realtime Database*. https://firebase.google.com/docs/database.

[17] Google. (n.d.). *Flutter Documentation*. https://docs.flutter.dev.

[18] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.